CSL7110 – Big Data Analytics
Assignment 2 – MinHash and Locality Sensitive Hashing

Name: Basu Singh
Roll Number: M24AID008
Course: CSL7110 – Big Data Analytics

GitHub Repository:
https://github.com/Yoursuperherobasu/M24AID008_CSL7110-Minhash-LSH-Assignment.git

Google Colab Notebook:
https://colab.research.google.com/drive/1wHfbJtEOKMOuKq029wM2IJWvUQDFlMuX?usp=sharing

1. Introduction

Detecting similarity between large documents is an important task in big data applications such as plagiarism detection, duplicate document identification, and web page clustering. When the number of documents is very large, computing exact similarity between all document pairs becomes computationally expensive.

To address this problem, techniques such as **MinHash** and **Locality Sensitive Hashing (LSH)** are used. MinHash provides an efficient approximation of the Jaccard similarity between document sets, while LSH helps in quickly identifying candidate pairs of similar documents without comparing every pair directly.

In this assignment, the MinHash and LSH techniques were implemented to estimate document similarity. Shingles were generated from the input documents, exact Jaccard similarities were computed, and then MinHash signatures were used to approximate similarity efficiently. Finally, LSH was applied to detect candidate document pairs that are likely to be similar.

2. Dataset Description

The dataset provided for this assignment consists of four text documents:

- D1.txt
- D2.txt
- D3.txt
- D4.txt

Each document contains textual content that is used to compute similarity between documents. The goal is to determine how similar the documents are using different shingling techniques and MinHash approximation.

The documents were loaded into a Google Colab environment, where the text was preprocessed by converting it to lowercase and normalizing whitespace. This preprocessing ensures consistent shingle generation and similarity computation.

3. Shingling

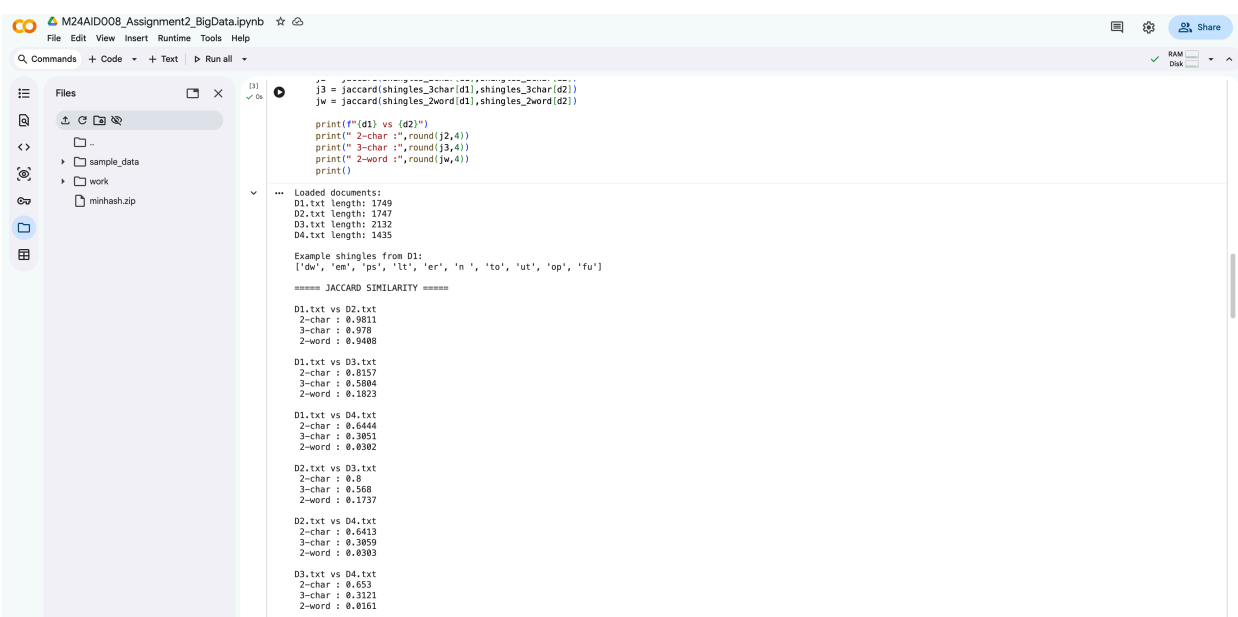
Shingling is a technique used to convert a document into a set of contiguous substrings called shingles or k-grams. These shingles represent the document content in a structured form that allows similarity computation using set operations.

In this assignment, three different types of shingles were generated:

- **2-character shingles**
- **3-character shingles**
- **2-word shingles**

Character-based shingles capture fine-grained textual similarity, while word-based shingles capture similarity at the semantic level.

After generating shingles for each document, the resulting sets were used to compute similarity using the Jaccard coefficient.



4. Jaccard Similarity

To measure the similarity between two documents, the **Jaccard similarity coefficient** was used.

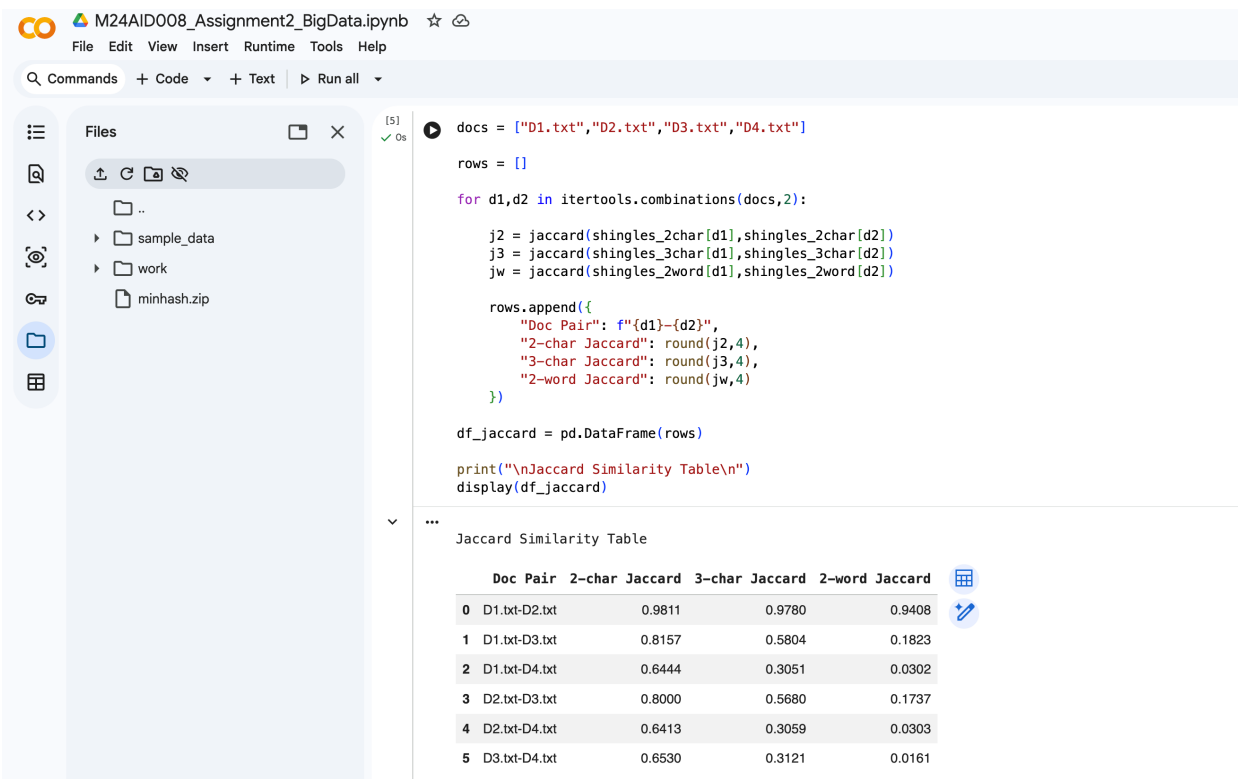
The Jaccard similarity between two sets A and B is defined as:

$J(A,B) = |A \cap B| / |A \cup B|$

Where:
- $|A \cap B|$ represents the number of common elements between the two sets
- $|A \cup B|$ represents the total number of unique elements in both sets

Using this formula, Jaccard similarity was computed for all document pairs using each shingle type.

The following table shows the computed similarities.



	Doc Pair	2-char Jaccard	3-char Jaccard	2-word Jaccard
0	D1.txt-D2.txt	0.9811	0.9780	0.9408
1	D1.txt-D3.txt	0.8157	0.5804	0.1823
2	D1.txt-D4.txt	0.6444	0.3051	0.0302
3	D2.txt-D3.txt	0.8000	0.5680	0.1737
4	D2.txt-D4.txt	0.6413	0.3059	0.0303
5	D3.txt-D4.txt	0.6530	0.3121	0.0161

5. MinHash Approximation

Computing exact Jaccard similarity for very large datasets can be computationally expensive. To overcome this limitation, the **MinHash algorithm** was used to approximate similarity between documents.

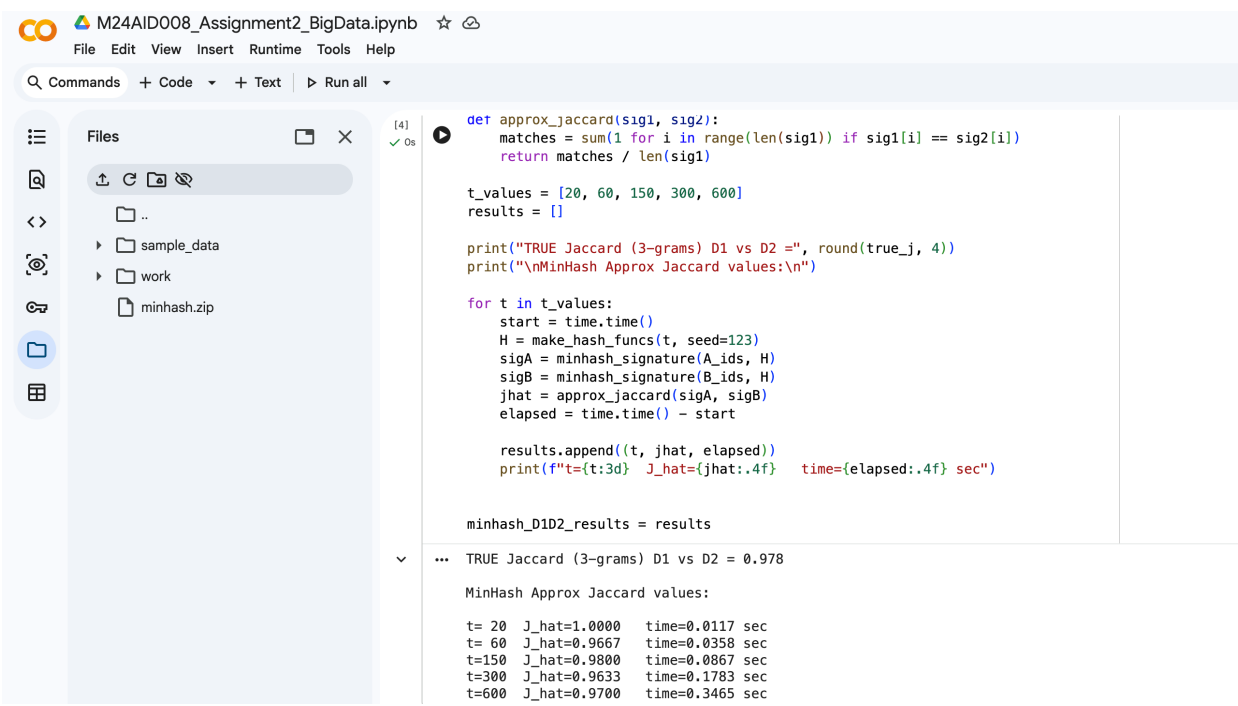
MinHash works by applying multiple hash functions to the shingle sets and storing the minimum hash value obtained for each function. These values form the MinHash signature of the document.

The similarity between two documents can then be estimated by comparing their MinHash signatures. The fraction of matching signature components approximates the Jaccard similarity.

In this assignment, MinHash signatures were generated using different numbers of hash functions:

- t = 20
- t = 60
- t = 150
- t = 300
- t = 600

As the number of hash functions increases, the MinHash estimate becomes closer to the true Jaccard similarity.



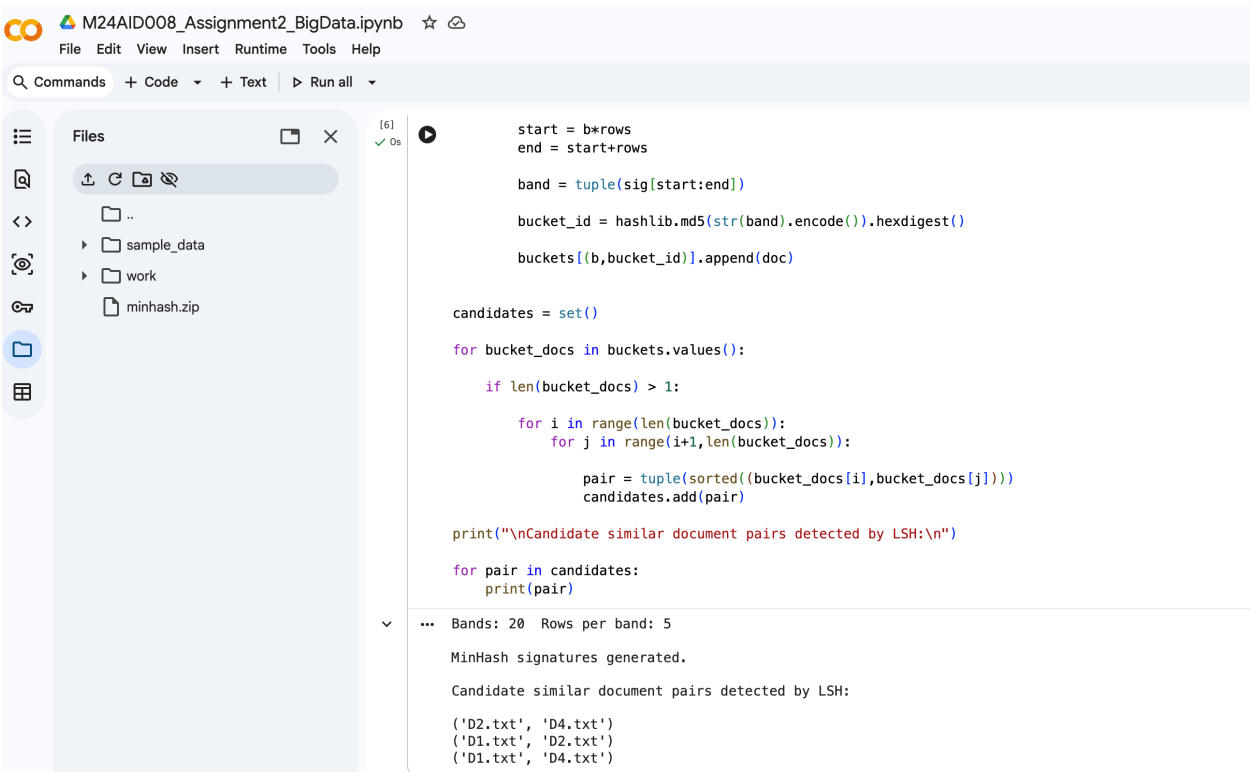
6. Locality Sensitive Hashing (LSH)

While MinHash reduces the cost of similarity estimation, comparing every pair of documents is still inefficient for very large datasets. Locality Sensitive Hashing (LSH) addresses this problem by grouping similar signatures into buckets.

The MinHash signature matrix is divided into several bands, and documents whose signatures match within a band are placed in the same bucket. Documents that share a bucket are considered candidate pairs and are then checked for similarity.

In this assignment, the MinHash signatures were divided into multiple bands, and the LSH algorithm was applied to identify candidate document pairs that are likely to be similar.

This approach significantly reduces the number of pairwise comparisons required in large datasets.



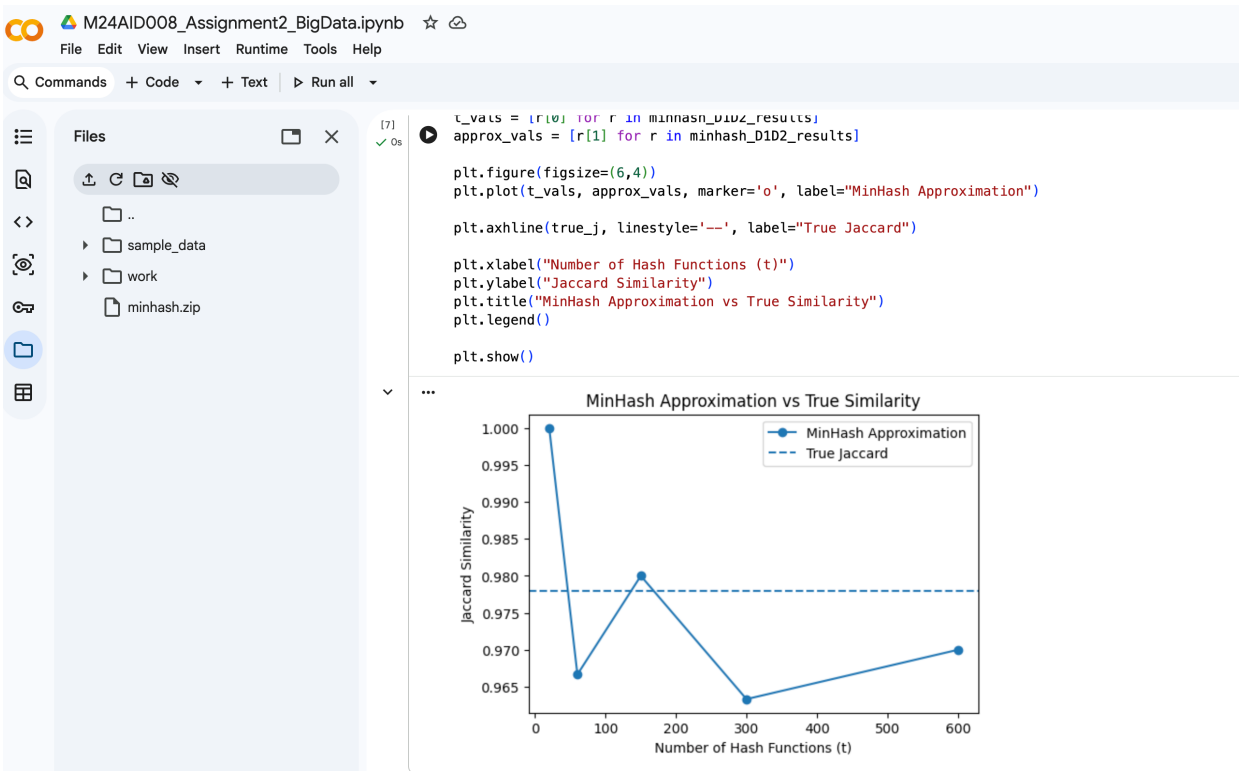
7. MinHash Accuracy Analysis

To evaluate the effectiveness of the MinHash algorithm, the approximate similarity values obtained from MinHash were compared with the true Jaccard similarity.

A plot was generated showing the relationship between the number of hash functions and the estimated similarity value. The results show that as the number of hash functions increases, the

MinHash approximation becomes more accurate and converges toward the true Jaccard similarity.

This demonstrates that increasing the number of hash functions improves the reliability of MinHash estimates.



8. Conclusion

In this assignment, document similarity detection was implemented using shingling, Jaccard similarity, MinHash, and Locality Sensitive Hashing.

Different shingle representations were used to convert documents into sets, and exact Jaccard similarities were computed for all document pairs. The MinHash algorithm was then applied to approximate similarity efficiently using hash signatures. Finally, Locality Sensitive Hashing was used to identify candidate document pairs with reduced computational cost.

The results demonstrate that MinHash provides a reliable approximation of Jaccard similarity, and LSH enables efficient similarity detection in large document collections. These techniques are widely used in large-scale information retrieval systems and duplicate detection applications.